Gyrosensor glove

Student: Bănică Ninel-Valentin
Group: 4LF521A – IEC III
Date: 28.05.2025
Course: Computer Interfaces and Peripherals

Contents

Chapter 1: Objectives of the paper 3

Chapter 2: Component description 3

1. Arduino Uno and Plusivo Boards 3
2. MPU 6050 Accelerometer and Gyro-sensor 4
3. HC-05 Bluetooth module 5

AT MODE 6

4. BC1604A LCD 16x4 9

Chapter 3: Description of work 11

Introduction of project 11

1. Gyro-sensor glove setup 11

Numerical processing of the gyro-sensor data 13

2. Game screen setup 15

Chapter 4: Working with registers 19

Chapter 5: Code 22

Gyro-sensor glove 22

glove.ino 22

DataReader.cpp 24

DataReader.h 28

Game screen 30

screen.ino 30

gamemanager.cpp 30

gamemanager.h 41

worm.cpp 42

worm.h 45

segment.cpp 46

segment.h 47

Vector.cpp 47

Vector.h 49

Chapter 1: Objectives of the paper

Aim of the project

This project combines sensor detection of hand rotation using a gyro-sensor, with wireless data transmission through Bluetooth and control of real time data displayed on an LCD.

Does this solve any real-life problem?

The components used in this project could be used to implement control of larger scale electronic devices like a robotic arm, a motorized wheelchair, etc. through simple hand gestures.

Chapter 2: Component description

1. Arduino Uno and Plusivo Boards

Fig. 2.1.1. Arduino UNO board

Fig. 2.1.2. Plusivo board

ATTENTION: The Plusivo board is just an alternative board of the Arduino board. One could use 2 Arduino boards instead with no additional configurations/issues.

2. MPU 6050 Accelerometer and Gyro-sensor

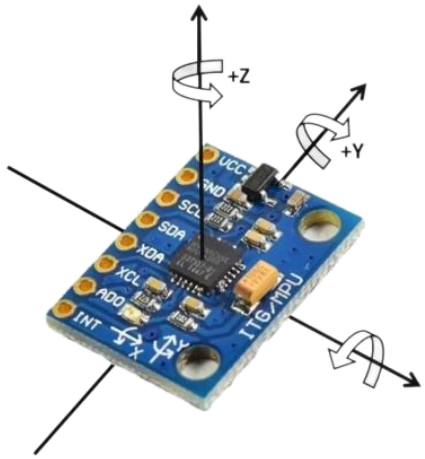


Fig. 2.2.1. MPU 6050 and the rotation axis of the inner gyro-sensor

The module MPU 6050 offers analog values based on the following parameters:

- Rotation on the X, Y, Z axis.
- Acceleration on the X, Y, Z axis.
- (If the module is an MPU 6050/9050) it can offer magnetometer(*compass*) data.

Characteristic	**Value**
Supply voltage	5V
Protocol	I2C
Output type	Analog

Table 2.2.1. Characteristics of the MPU 6050

To use this module, an external library is required:

MPU6050.h	https://docs.arduino.cc/libraries/mpu6050/

Table 2.2.2. External library needed to use MPU6050

3. HC-05 Bluetooth module

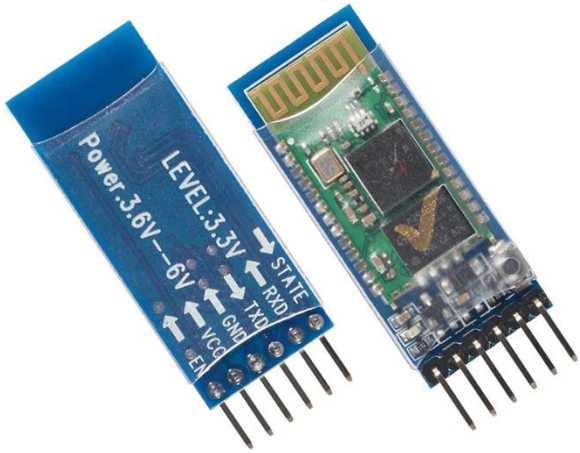


Fig. 2.3.1. HC-05 Bluetooth module

Characteristic	Value
Supply voltage	5V
Protocol	UART
Output type	Digital

Table 2.3.1. Characteristics of an HC-05

The connections of such a module need to be carefully done using a voltage divider on the RXD output pin as in the figure below:

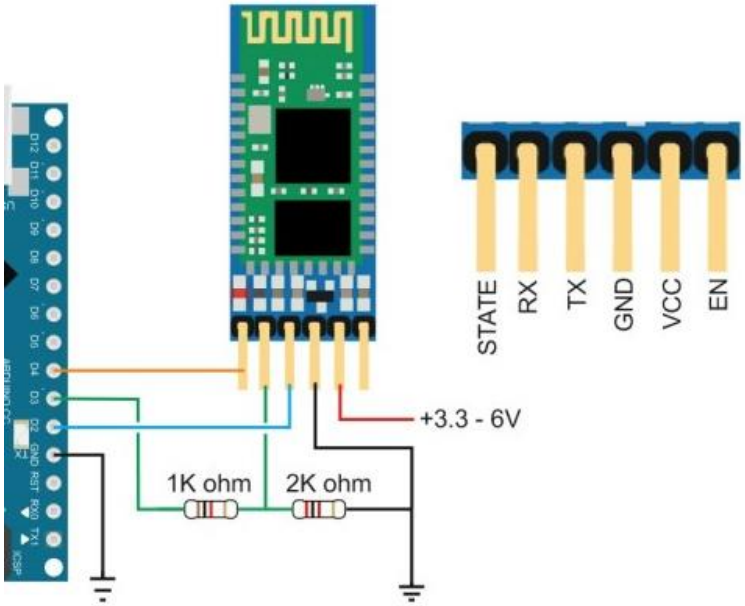


Fig. 2.3.2. HC-05 connections (with pair state confirmation)

IMPORTANT: If using multiple HC-05 modules which should communicate with each other, a connection of type sender-receiver is necessary. This can be done by configuring the internal settings of the module.

The module has LEDs which blink according to its mode:

LED action	Module mode
Blinking quickly each ~1s	Trying to pair
Blinking quickly each ~5s	Paired
Blinking slowly each ~2s	AT Mode

Table 2.3.2. Operating modes

To use this module, an library is required:

SoftwareSerial.h	https://docs.arduino.cc/learn/built-in-libraries/software-serial/

Table 2.3.3. Library needed to use HC-05

AT MODE

This is the module's configuration mode. To enter AT mode, the **EN** pin needs to be set to **HIGH** on startup. There are a few ways to do this:

1. Connect the **EN pin** to **VCC** before starting the module.
2. (***RECOMMENDED***) Hold the button above the **EN pin** (*shown in the figure below*) before starting the module, then wait until the LED marks entering AT Mode as indicated in the table above, after which the button can be released.

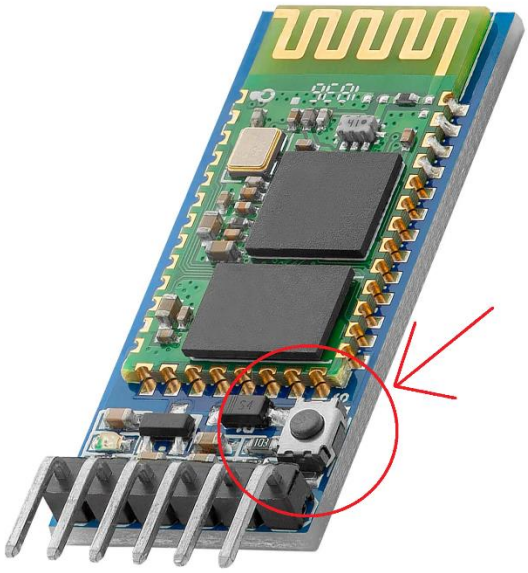


Fig. 2.3.3. HC-05 AT Mode enable button

Once the module is in AT Mode, it can be configured using the following code snippet:

```
#include <SoftwareSerial.h>

SoftwareSerial btSlave(10, 11); // RX, TX pins for HC-05

void setup() {
  Serial.begin(9600);         // Open serial communication with computer
  btSlave.begin(38400);       // Open serial communication with HC-05 (check baud
rate)
  Serial.println("Enter AT commands:");
}

void loop() {
  if (btSlave.available()) {    // If HC-05 sends data
    Serial.write(btSlave.read());  // Send data to serial monitor
  }
  if (Serial.available()) {       // If serial monitor sends data
    btSlave.write(Serial.read());  // Send data to HC-05
  }
}
```

Explanation:

- The code above uses the **SoftwareSerial.h** library to use the digital pins 10 & 11 as UART connections to communicate between the module and the Arduino.
- The Serial monitor uses 9600 baud rate by default. The module by default uses 38400 baud rate.
- The code in the loop takes all the text sent through the Serial console and all the text received from the module and prints both in the **Serial monitor**.

And the following connections:

HC-05	Arduino
VCC	VCC
GND	GND
TXD	D10
RXD	D11 (through a voltage divider)

Table 2.3.4. Connections for configuring an HC-05

Once the connections have been done, the code was uploaded and the module entered AT mode, the Arduino IDE needs to have the Serial monitor set as follows:

- Both NL & CR
- 38400 baud

Fig. 2.3.4. Serial monitor settings

Now, the following commands can be used in the Serial monitor as follows:

1. AT -> prints OK if the module is in AT mode and is reading the console
2. _OPTION_? -> prints the value already stored in the mentioned **_OPTION_** field
3. _OPTION_=_VALUE_ -> sets the **_VALUE_** to the mentioned **_OPTION_** field

Here is a list with a few of the available commands:

Command	Meaning	Default value
AT+NAME	Name displayed to other devices	hc05
AT+ADDR	Adress used to pair modules	unique on each device
AT+UART	Baud rate	38400
AT+ROLE	0 = Slave | 1 = Master	0
AT+RESET	Reset and exit AT mode	-
AT+ORGL	Reset to factory settings	-
AT+PSWD	Password to connect to the module from another device	1234 **OR** 0000
AT+BIND	AT+ADDR to which this module tries to pair by itself	none

Table 2.3.5. Important commands of AT mode

To properly make two HC-05 modules pair to each other:

1. Enter AT mode in both and get the AT+ADDR? of both.
2. Set first as Master AT+ROLE=1
3. Bind the first module to the second module's address AT+BIND=ADDR2
4. Set the 2nd as Slave AT+ROLE=0 (*optional as the modules are set to Slave by default*)
5. Bind the second module to the first module's address AT+BIND=ADDR1
6. Useful tip:
    - Modify the name and passwords of both modules

4. BC1604A LCD 16x4

Fig. 2.4.1. LCD 16x4 shield module

To use this module, an library is required:

LiquidCrystal.h	https://docs.arduino.cc/libraries/liquidcrystal/

Table 2.4.1. Library needed to use the LCD shield

Characteristic	Value
Supply voltage	5V
Protocol	Parallel interface protocol
Output type	Digital

Table 2.4.2. Characteristics of an LCD shield

For this project I have used a special feature of the LCD shield to create custom characters to display a game of Snake chasing fruits.

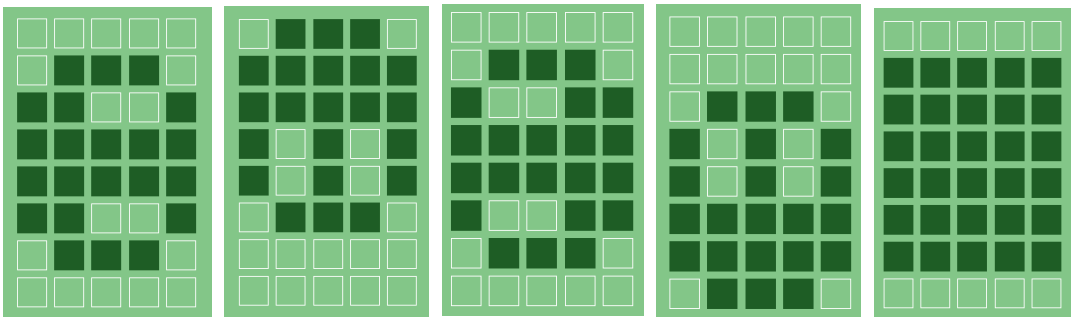


Fig. 2.4.2. Snake head orientations (West, North, East, South) and body segment

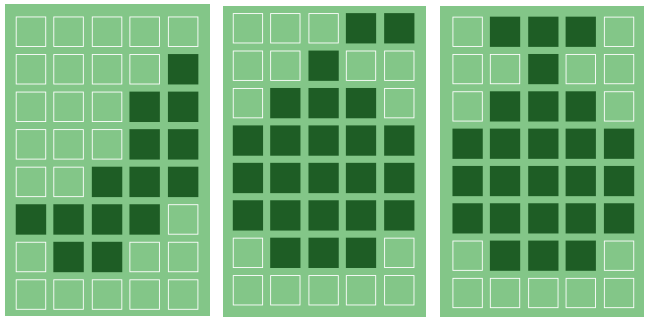


Fig. 2.4.3. Fruit characters

These characters are a vector of bytes and each 1 represents a turned-on pixel.
Example: The first banana fruit character has the following binary representation in code:

```
const byte fruit[8] = {
  0b00000,
  0b00001,
  0b00011,
  0b00011,
  0b00111,
  0b11110,
  0b01100,
  0b00000
};
```

The connections should be made as follows:

LCD	Arduino
VCC	VCC
GND	GND
VO	Connected to the middle port of a potentiometer of **10 kΩ**
RS	D7
RW	GND
E	D6
DB0	-
DB1	-
DB2	-
DB3	-
DB4	D5
DB5	D4
DB6	D3
DB7	D2
LEDA	VCC
LEDK	GND

Table 2.4.3. Connections for using the LCD shield

Chapter 3: Description of work

Introduction of project

As mentioned before the project itself is composed of two individual Arduinos and their own electrical systems:

1. Gyro-sensor glove setup
2. Game screen setup

1. Gyro-sensor glove setup

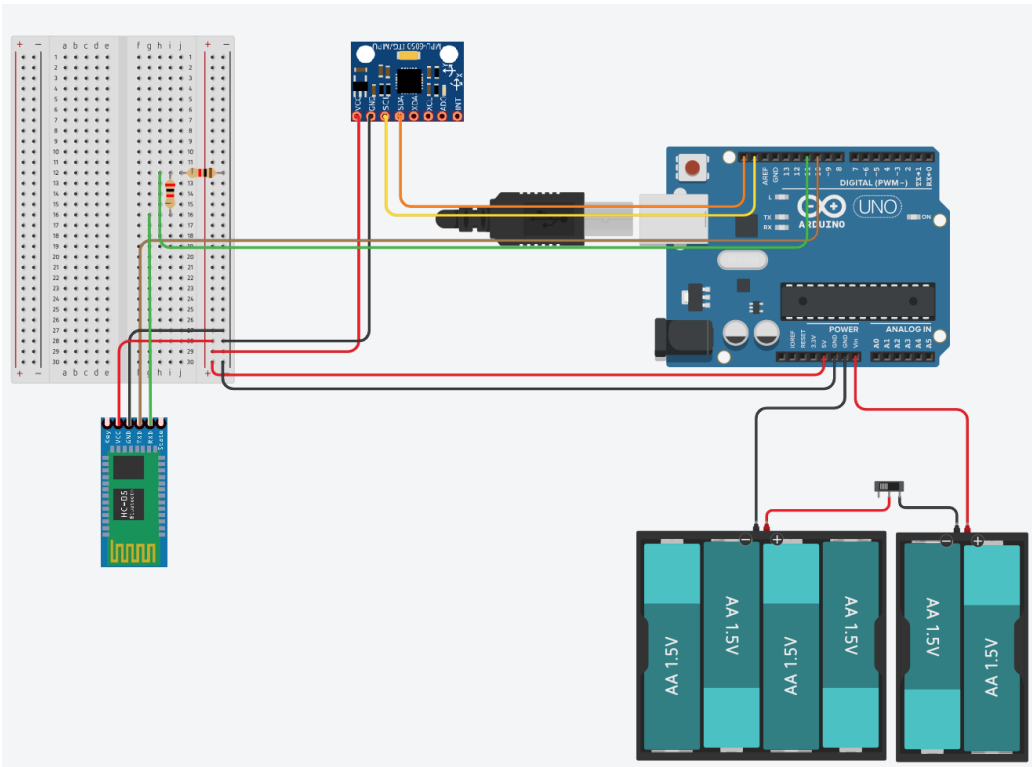


Fig. 3.1.1. The gyro-sensor glove electrical diagram (made using Tinkercad.com)

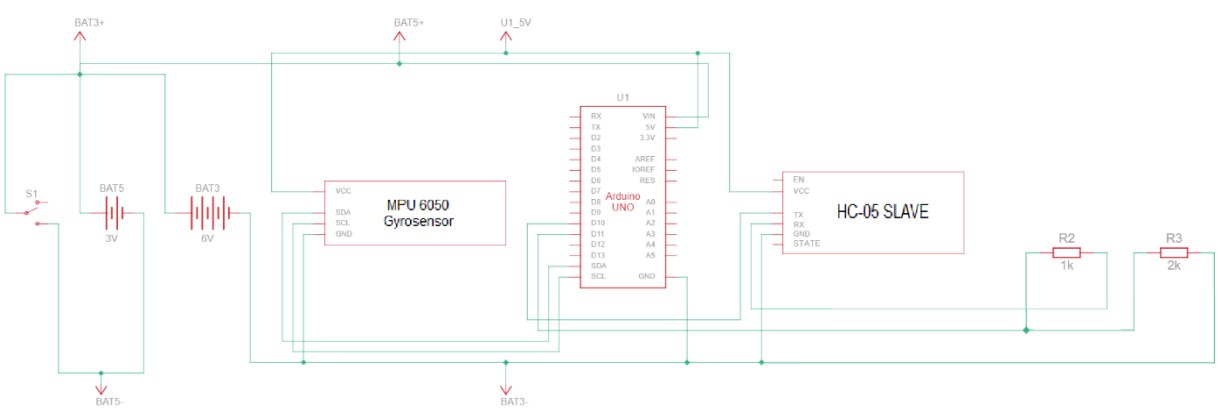


Fig. 3.1.2. The gyro-sensor glove electrical schematic (made using Tinkercad.com)

The Arduino here is powered by an external **6*1.5V** battery setup, providing a total of **9V**, perfect for the supply of the board as it has an internal voltage regulator. The two battery sockets are connected using a switch to allow the user to turn it on and off at will.

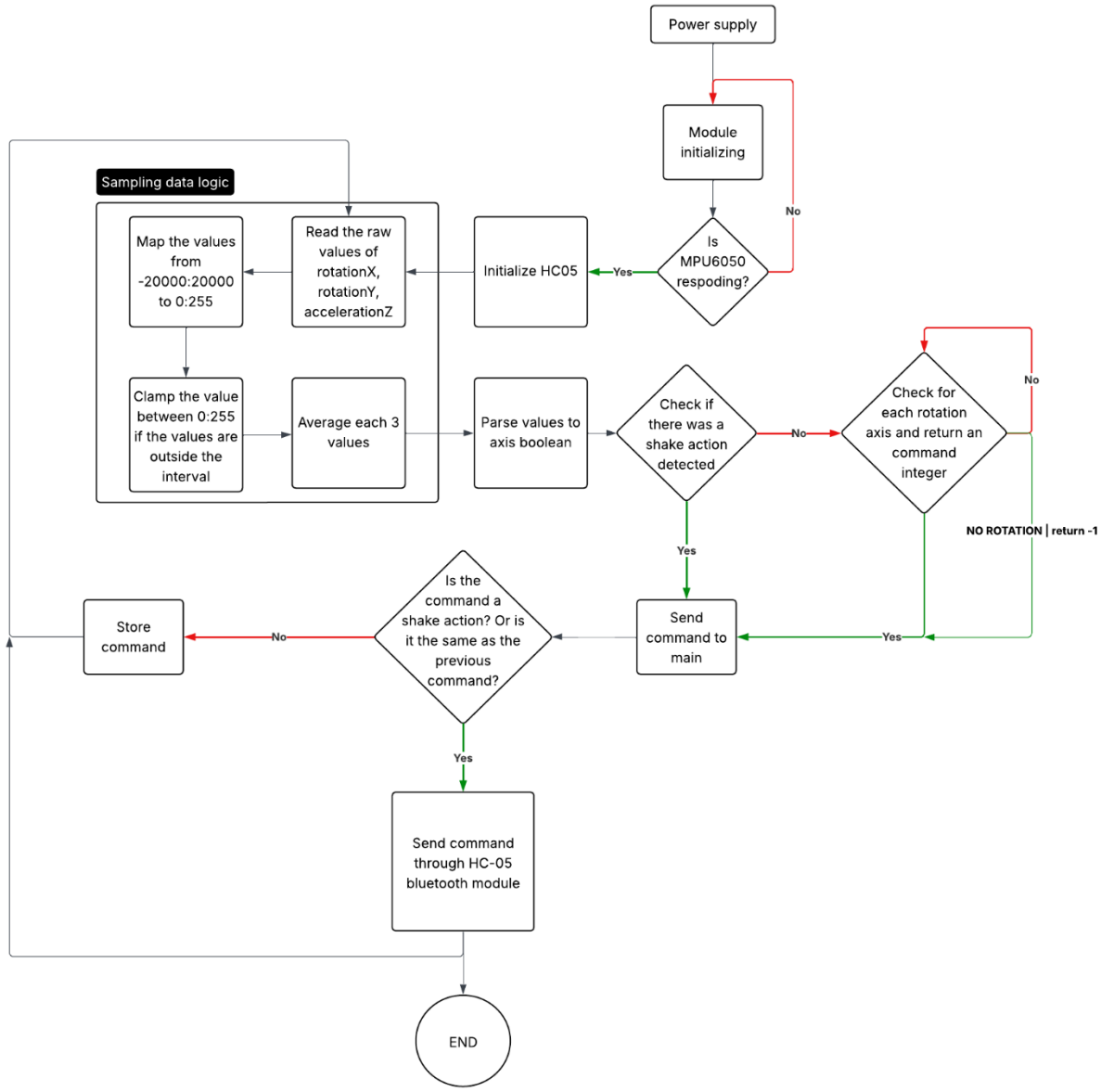


Fig. 3.1.3. The gyro-sensor glove block diagram (made using lucid.app/lucidchart/)

The gyro-sensor is used to detect motion and rotation as described below, after which the signal is processed and parsed to a command integer and sent via the HC-05 to the 2nd system.

Numerical processing of the gyro-sensor data

The steps to read and process the numbers are as follows:

1. Read the data from the MPU6050 using the library methods. These will return numbers between -20000:20000 approximately.
   (*These values have been determined though testing*)
2. The numbers are mapped to numbers from the interval 0:255 to easily process and monitor them.
3. The mapped values are clamped between 0:255 because some values may spike over the initial interval of -20000:20000.
4. These clamped values are once again mapped to a (-1:0:1) mapping meaning (RIGHT/DOWN : NO_ROTATION : LEFT/UP).

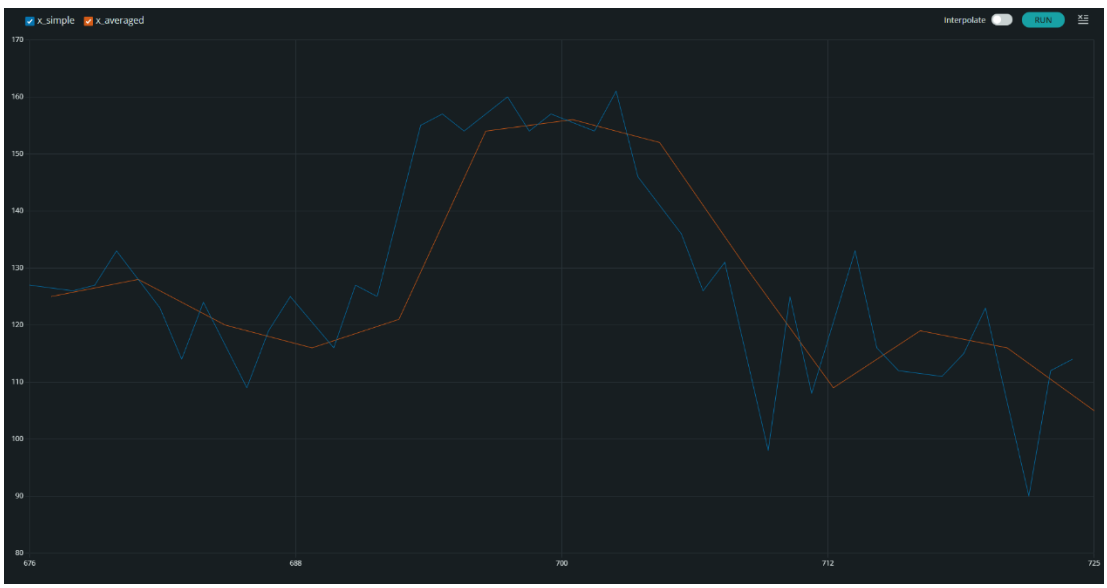


Fig. 3.1.4. Graphical representation of what effect the numerical averaging has on the signal

As presented above, the blue signal is each value reading, which makes the signal VERY noisy.
On the same chart, the orange one is ***much smoother*** but **slightly lagged**.

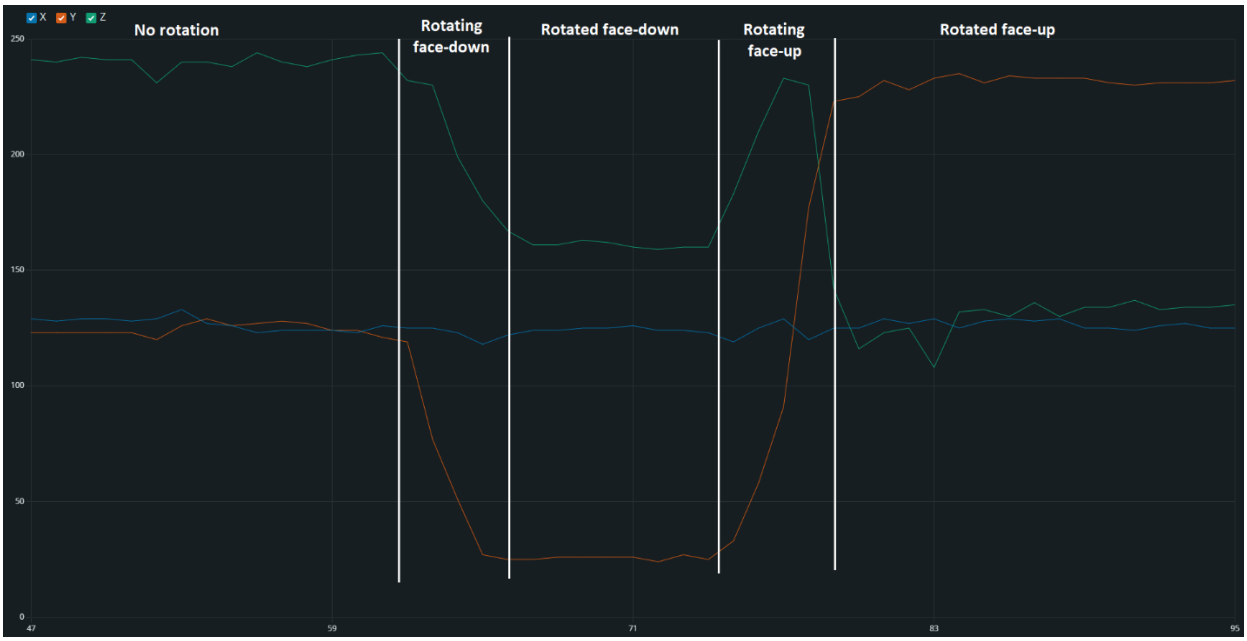


Fig. 3.1.5. Graphical representation of hand DOWN/UP movement

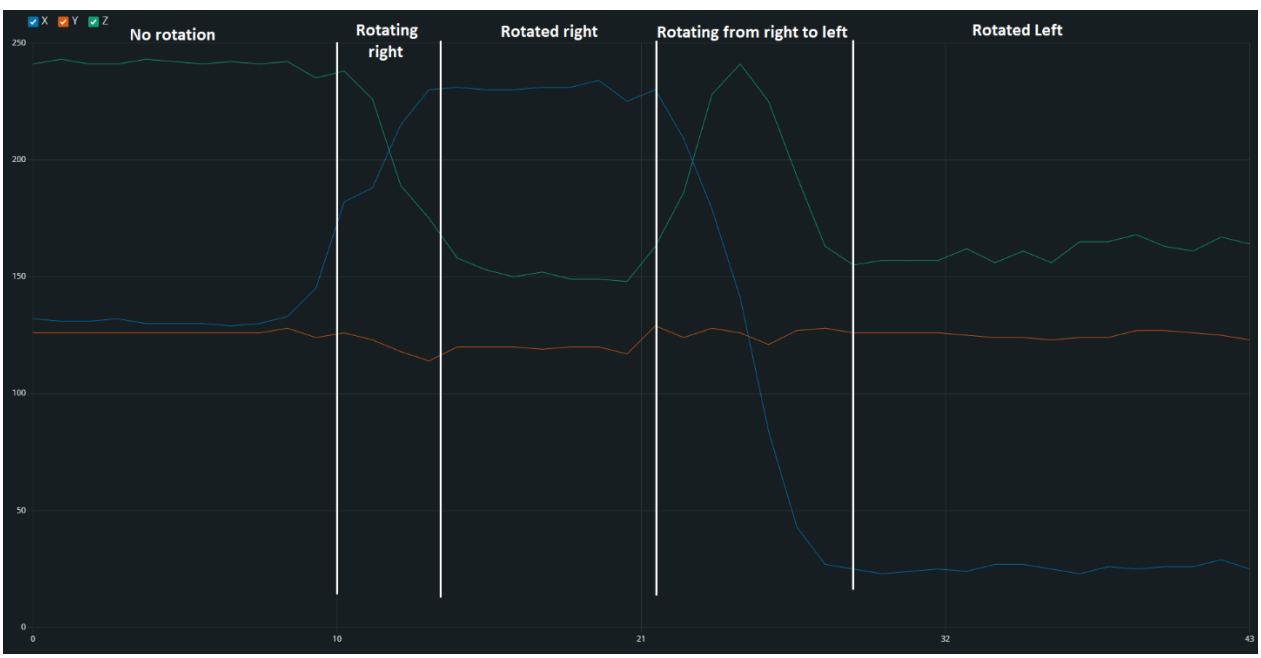


Fig. 3.1.6. Graphical representation of hand RIGHT/LEFT movement

The images above show the 3 axis rotation readings during different gyro-sensor rotations.

With the help of such graphical representations it can easily be interpreted different axis between different thresholds mean a certain rotation. This allowed parsing the values to the three directional mapping mentioned before.

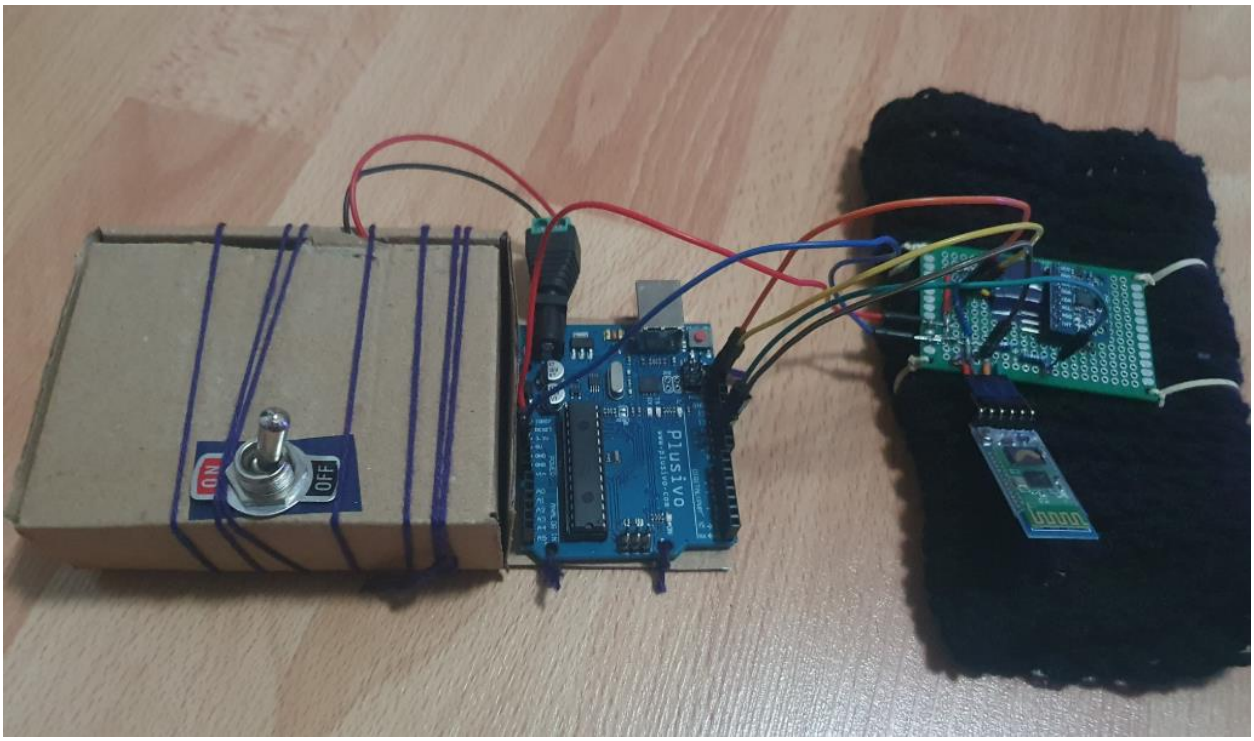


Fig. 3.1.7. Actual setup of the glove

2. Game screen setup

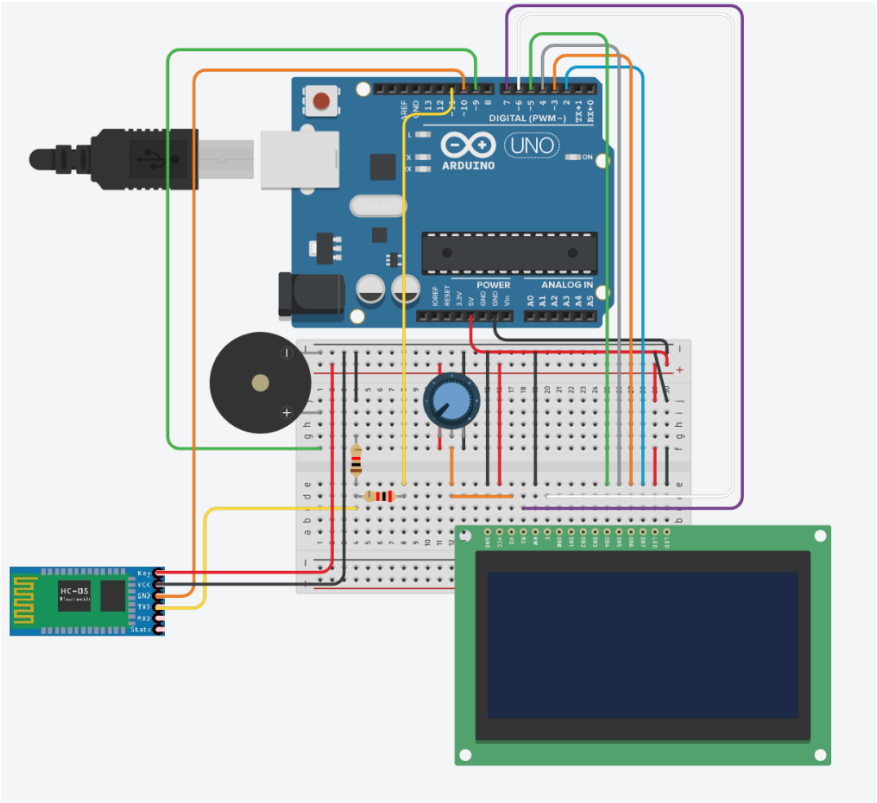


Fig. 3.2.1. The game screen electrical diagram (made using Tinkercad.com)

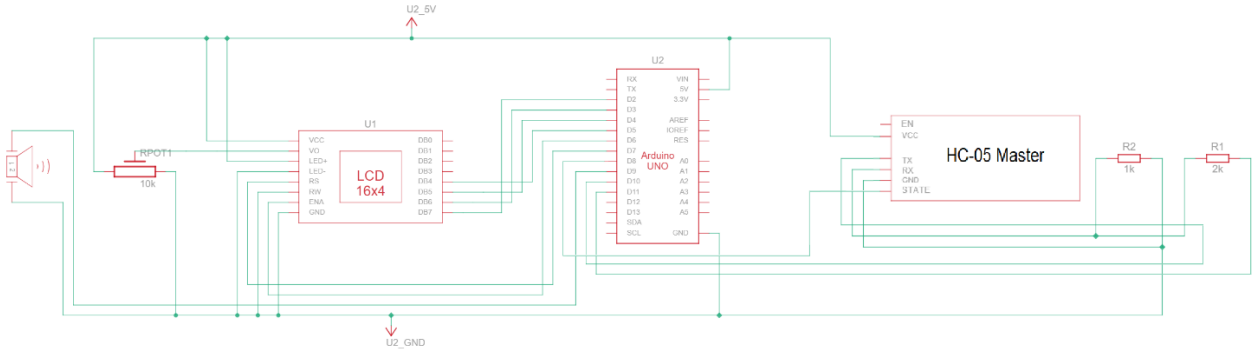


Fig. 3.2.2. The game screen electrical schematic (made using Tinkercad.com)

This system is running a rather complex program in the background to simulate an entire video game slightly described below:

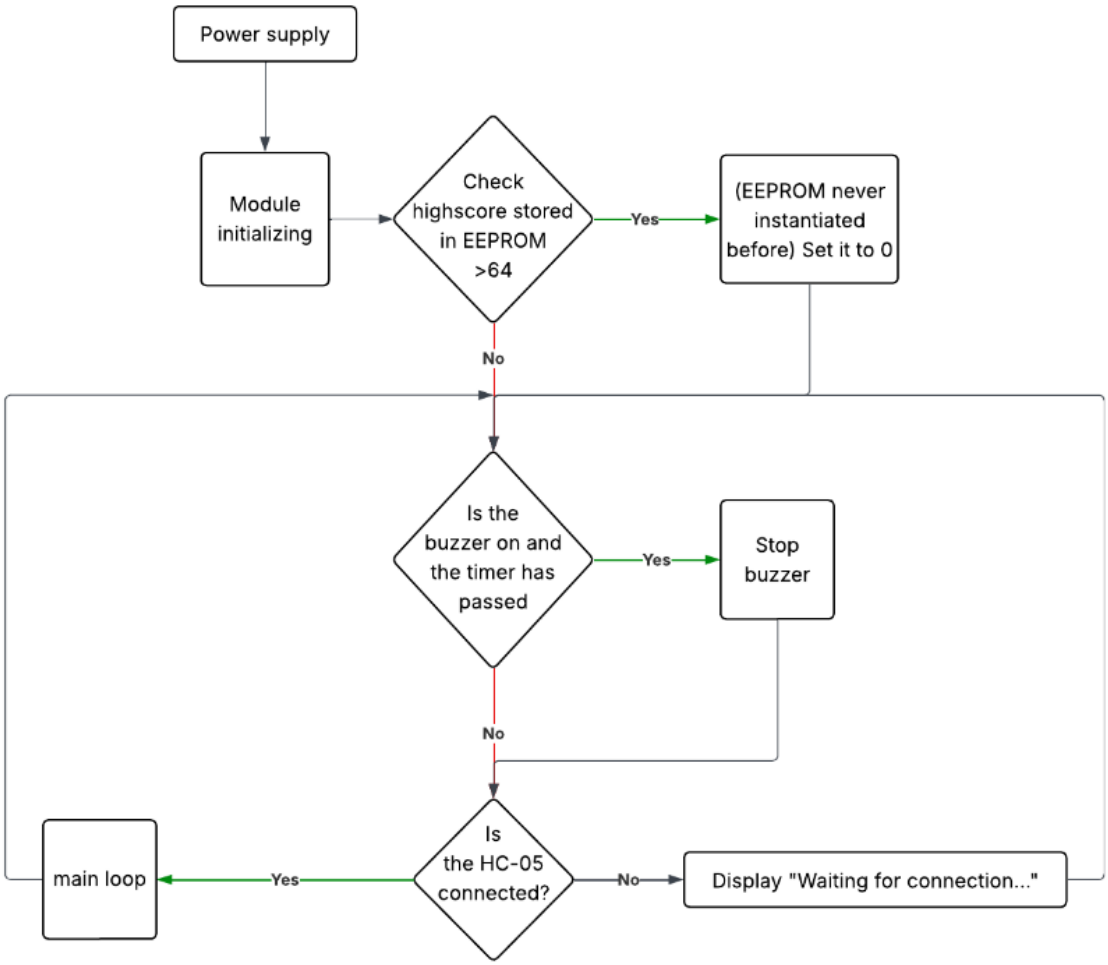


Fig. 3.2.3. The game screen block diagram (made using lucid.app/lucidchart/)

The figure above is the main block diagram of the circuit. It uses the EEPROM Arduino memory to hold high-score even after the board is powered off.

The buzzer is controlled using registers, therefore each loop iteration a check for a Boolean variable is needed to properly stop the buzzer after a certain period of time.

The "**main loop**" block is described by the block diagram below:

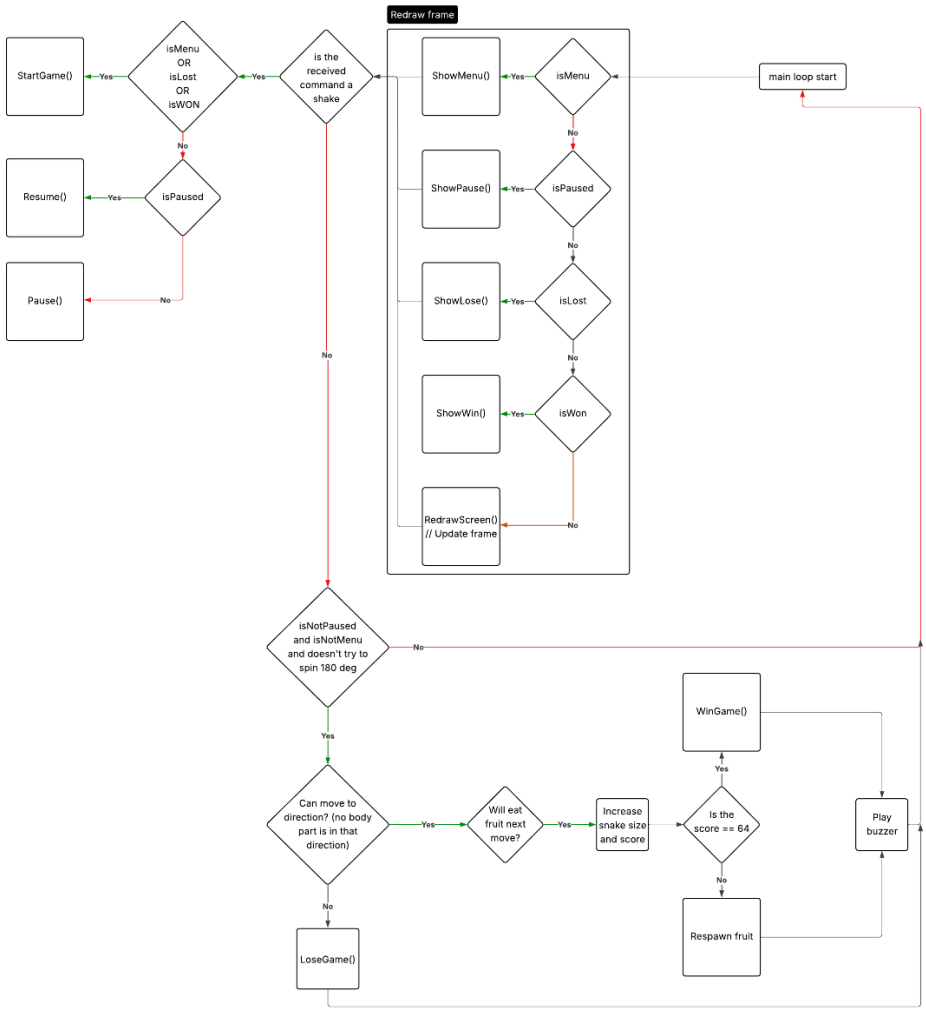


Fig. 3.2.4. The main loop block diagram (made using lucid.app/lucidchart/)

The different methods shown don't impact the game logic much other than displaying a certain menu or resuming from the menu to the game.

Below a table with each state of the LCD:

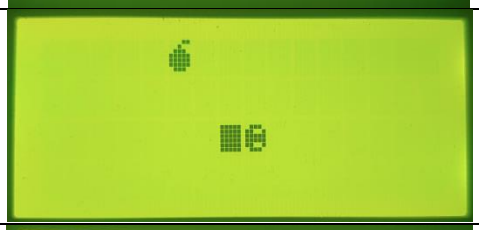
Identifier	Meaning	Screen picture
Waiting for connection	This is an animated frame that is shown while the HC-05 modules are trying to pair with each other.	
Start menu	Main menu. Waiting for confirmation to start the game.	
Game frame	Game display. Waiting for commands to move.	
Pause menu	During game, a shake will trigger the *pause menu* to display.	
Lose screen	Losing the game will display the *Lose screen*.	
Win screen	Winning the game will display the *Win screen*.	

Table 3.2.1. All display modes of the LCD

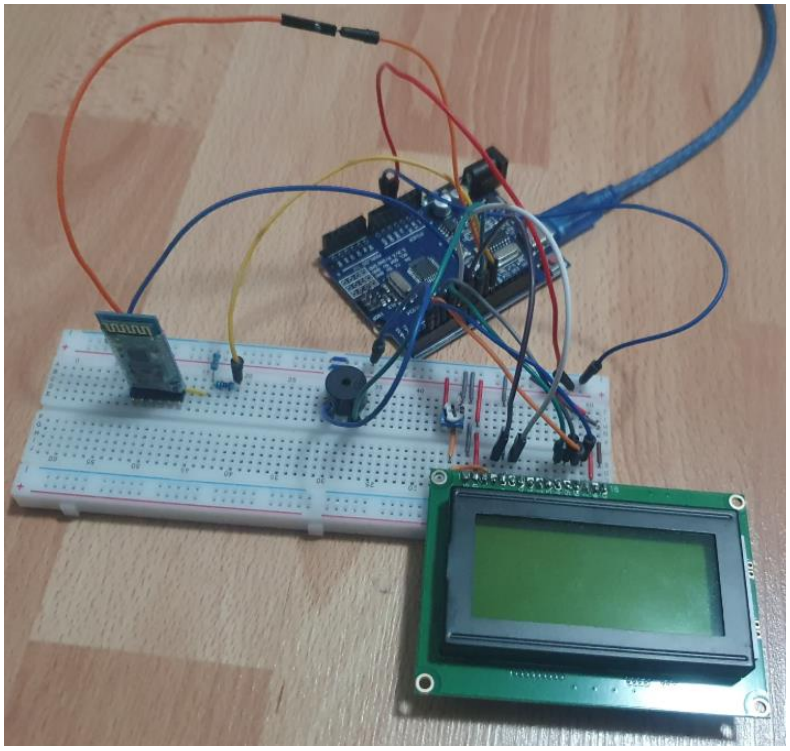


Fig. 3.2.5. Actual setup of the display

Chapter 4: Working with registers

I have defined a way to use a buzzer like the already existing tone() method by using Timer1 in CTC mode to compute a frequency needed for the buzzer to be on.

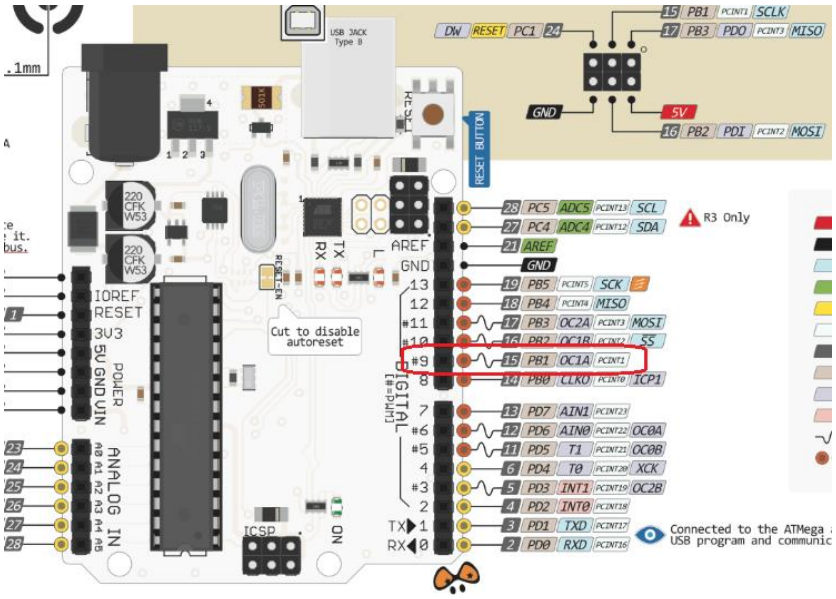


Fig. 4.1.1. Arduino pinout : digital pin 9 - OC1A

The buzzer is set on pin 9 (OC1A), therefore the OC1A needs to be toggled on CTC mode of Timer1.

- First things first, set the TCCR1A and TCCR1B registers to 0 to reset potential previous settings.

```
TCCR1A = 0;  // Clear any previous settings
TCCR1B = 0;  // Clear any previous settings
```

- Set TCCR1A to toggle pin 9 on compare match of CTC Timer1.

15.11.1 TCCR1A – Timer/Counter1 Control Register A

Bit	7	6	5	4	3	2	1	0	
(0x80)	COM1A1	COM1A0	COM1B1	COM1B0	–	–	WGM11	WGM10	TCCR1A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Table 15-2. Compare Output Mode, non-PWM

COM1A1/COM1B1	COM1A0/COM1B0	Description
0	0	Normal port operation, OC1A/OC1B disconnected.
0	1	Toggle OC1A/OC1B on compare match.
1	0	Clear OC1A/OC1B on compare match (set output to low level).
1	1	Set OC1A/OC1B on compare match (set output to high level).

Fig. 4.1.2. TCCR1A register and compare output mode

```
// Toggle OC1A on Compare Match
TCCR1A |= (1 << 6);  // (1 << COM1A0);
```

- Set the timer to CTC mode

15.11.2 TCCR1B – Timer/Counter1 Control Register B

Bit	7	6	5	4	3	2	1	0	
(0x81)	ICNC1	ICES1	–	WGM13	WGM12	CS12	CS11	CS10	TCCR1B
Read/Write	R/W	R/W	R	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Fig. 4.1.3. TCCR1B register

Table 15-5. Waveform Generation Mode Bit Description[1]

Mode	WGM13	WGM12 (CTC1)	WGM11 (PWM11)	WGM10 (PWM10)	Timer/Counter Mode of Operation	TOP	Update of OCR1x at	TOV1 Flag Set on
0	0	0	0	0	Normal	0xFFFF	Immediate	MAX
1	0	0	0	1	PWM, phase correct, 8-bit	0x00FF	TOP	BOTTOM
2	0	0	1	0	PWM, phase correct, 9-bit	0x01FF	TOP	BOTTOM
3	0	0	1	1	PWM, phase correct, 10-bit	0x03FF	TOP	BOTTOM
4	0	1	0	0	CTC	OCR1A	Immediate	MAX
5	0	1	0	1	Fast PWM, 8-bit	0x00FF	BOTTOM	TOP
6	0	1	1	0	Fast PWM, 9-bit	0x01FF	BOTTOM	TOP
7	0	1	1	1	Fast PWM, 10-bit	0x03FF	BOTTOM	TOP
8	1	0	0	0	PWM, phase and frequency correct	ICR1	BOTTOM	BOTTOM
9	1	0	0	1	PWM, phase and frequency correct	OCR1A	BOTTOM	BOTTOM
10	1	0	1	0	PWM, phase correct	ICR1	TOP	BOTTOM
11	1	0	1	1	PWM, phase correct	OCR1A	TOP	BOTTOM
12	1	1	0	0	CTC	ICR1	Immediate	MAX
13	1	1	0	1	(Reserved)	–	–	–
14	1	1	1	0	Fast PWM	ICR1	BOTTOM	TOP
15	1	1	1	1	Fast PWM	OCR1A	BOTTOM	TOP

Note: 1. The CTC1 and PWM11:0 bit definition names are obsolete. Use the WGM12:0 definitions. However, the functionality and location of these bits are compatible with previous versions of the timer.

Fig. 4.1.4. Waveform generation mode needed for CTC mode

```
// CTC mode
TCCR1B |= (1 << 3);  // (1 << WGM12);
```

- A prescaler of 8 will be used to compute the value to which the Timer will be compared.

$$\frac{16MHz}{8 * 2 * 600Hz} - 1 = 1666$$

Where:

- 16MHz = the frequency of the microcontroller
- 8 = the prescaler value
- 2 = half of the value as the buzzer will be on half of the period (50% duty cycle)
- 600Hz = the frequency chosen through testing which sounded best for this application.
- -1 = the timer counts from 0 therefore the value will be -1

Table 15-6. Clock Select Bit Description

CS12	CS11	CS10	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	$clk_{I/O}$/1 (no prescaling)
0	1	0	$clk_{I/O}$/8 (from prescaler)
0	1	1	$clk_{I/O}$/64 (from prescaler)
1	0	0	$clk_{I/O}$/256 (from prescaler)
1	0	1	$clk_{I/O}$/1024 (from prescaler)
1	1	0	External clock source on T1 pin. Clock on falling edge.
1	1	1	External clock source on T1 pin. Clock on rising edge.

Fig. 4.1.5. Prescaler values for TCCR1B

```
TCCR1B |= (1 << 1);  // (1 << CS11);
```

- Mentioned above in *Fig. 4.1.4.*, the top of the CTC mode set is OCR1A, this needs to be set to the value computed above 1666.

```
// 600 Hz -> OCR1A = (2MHz / (2 * 600)) - 1 = 1666
OCR1A = 1666;
```

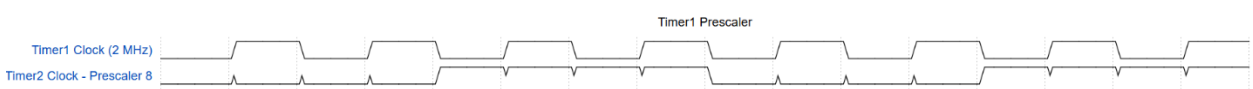


Fig. 4.1.6. Clock representation with and without prescaler

Chapter 5: Code

Gyro-sensor glove

glove.ino

```
#include <SoftwareSerial.h>  // library used for bluetooth communication
#include "DataReader.h"      // Custom class that manages the data processing and
reading

SoftwareSerial BTSerial(10, 11);  // 10:RX | 11:TX
DataReader* data;                 // Pointer to data object

int lastCmdSent = -1;  // Variable to store the last command sent to compare them
```

```
void setup() {
  data = new DataReader();  // Instantiate the DataReader object
  Serial.begin(9600);       // Serial communication for debugging
  BTSerial.begin(9600);     // Serial communication of bluetooth
  data->initialize();       // Initialization method for data

  if (!data->mpu.testConnection())                            // Check
communication with MPU6050
    Serial.println("Waiting for connection from MPU6050...");  // Display
debugging message
  while (!data->mpu.testConnection())                         // Wait for a
response frmo MPU6050
    ;
  Serial.println("Succesfully connected to MPU6050!");  // Display debugging
message for successfully connecting to MPU6050
}

void loop() {
  /*
    Info
    G = acceleration
    A = rotation

    acceleration.X    |   Vertical movement     |   Z axis
    acceleration.Z    |   Forward movement      |   X axis
    acceleration.Y    |   Horizontal movement   |   Y axis
  */

  int cmdToSend = data->computeAndSendAction();  // Compute average sample and
parse it to command index | This happens every 50 ms
  Serial.println(cmdToSend);
  if (cmdToSend == lastCmdSent                    // Check if the last 2 commands
were the same as to delay command spamming
      || cmdToSend == 4) {                        // Shake is detected only once
due to cooldowns and difficulty in triggering
    BTSerial.println(cmdToSend);                  // Send command to master HC-05
    lastCmdSent = -2;                             // Once a command has been sent,
reset the lastCmdSent to an integer which doesn't represent an actual command
  } else
    lastCmdSent = cmdToSend;  // Store last cmd

  /* 200ms delay
  *   50ms sampling delay
  * (200ms + 50ms) * 2 = 500ms
  *  500ms = reading delay of master
```

```
  */
  delay(200);
}
```

DataReader.cpp

```
#include "DataReader.h"

DataReader::DataReader() {}  // Empty constructor

void DataReader::initialize() {
  Wire.begin();      // Start communication with HC-05
  mpu.initialize();  // Start communication with MPU6050
}

void DataReader::readAndParseSampleData() {
  ax = mpu.getAccelerationX();  // Get rotation axis X      | The MPU6050
rotation and acceleration memories are inverted
  ay = mpu.getAccelerationY();  // Get rotation axis Y      | The MPU6050
rotation and acceleration memories are inverted
  rz = mpu.getRotationZ();      // Get acceleration axis Z  | The MPU6050
rotation and acceleration memories are inverted

  // To each point of the sample add the following:
  //  * the value read by the sensor for said axis
  //  * mapped from an interval from [-20000;20000] to an interval [0,255]
  //  * multiplied with the percentage needed to compute the average
  // [Example: for 3 samples per average, there would be 3 readings, each
multiplied with 0.33]

  avgAccDataSample.X += ClampValue(map(ax, -20000, 20000, 0, 255), 0, 255) * (1 /
(float)samplingCount);
  avgAccDataSample.Y += ClampValue(map(ay, -20000, 20000, 0, 255), 0, 255) * (1 /
(float)samplingCount);
  avgRotDataSample += ClampValue(map(rz, -20000, 20000, 0, 255), 0, 255) * (1 /
(float)samplingCount);

  delay(samplingDelay);  // apply the sampling delay between each reading
}

void DataReader::readAverageSample() {
  avgAccDataSample = {};                        // Reset average data sample to
0,0
  previousAdvgAccDataSample = avgRotDataSample;  // Store previous data sample
```

```
  avgRotDataSample = 0;                        // Create a new sample in which
the average will be computed


  for (int i = 0; i < samplingCount; i++)
    readAndParseSampleData();  // Iterate samplingCount and average the signal
}

int8_t DataReader::parseAccSampleToDirection(int16_t sampleValue) {
  // The absolute value of the difference between the sample and the step should
be lower than the error threshold
  // I.E. each value between [step-error;step+error] is considered part of the
step

  if (abs(directionUpperStep - sampleValue) <= directionErrorThreshold) return
1;   // Turn LEFT
  if (abs(directionLowerStep - sampleValue) <= directionErrorThreshold) return -
1;  // Turn RIGHT
  if (abs(directionMiddleStep - sampleValue) <= directionErrorThreshold) return
0;  // No rotation
}

Vector2 DataReader::readAccelerationAxis() {
  accelerationAxis = {};  // Reset the rotation axis to 0

  accelerationAxis.X = parseAccSampleToDirection(avgAccDataSample.X);  //
Transform the X axis from data signal to a -1:0:1 direction
  accelerationAxis.Y = parseAccSampleToDirection(avgAccDataSample.Y);  //
Transform the Y axis from data signal to a -1:0:1 direction

  return accelerationAxis;
}

bool DataReader::detectShake() {

  bool result = 0, spikeWasHit = 0;

  // compute the difference between 2 average signals
  spikeWasHit =
previousAvgAccDataSample
 // is the previous sample != 0 ?  (this is used to prevent the horn from
activating at start-up)
                &&
!accSpikeAlreadyDetected                                                //
can a new spike be registered
```

```
                && (abs(previousAvgAccDataSample - avgRotDataSample) >=
accDifferenceThreshold);  // has the hand rotated to generate the high signal

  if (spikeWasHit) {
    accSpikeAlreadyDetected = true;                                        //
Mark spike as registered to block registering others until another zero crossing
    accDifferenceFlags[previousAvgAccDataSample - avgRotDataSample < 0]++;  //
Increase flag for spike hit 0: positive(left-rotation) |OR| 1: negative(right-
rotation)
  }

  if (accDifferenceFlags[0] >= accDifferenceFlagThreshold[0] &&
accDifferenceFlags[1] >= accDifferenceFlagThreshold[1]) {  // Threshold has been
passed => shake event detected
    result = 1;
    // Reset threshold values
    accDifferenceFlags[0] = 0;
    accDifferenceFlags[1] = 0;
  }

  // If the spike is already detected but the spike threshold has been passed
  if (accSpikeAlreadyDetected
      && abs(previousAvgAccDataSample - avgRotDataSample) >=
accSpikeFlagResetThreshold)
    accSpikeAlreadyDetected = 0;  // Reset spike has already been detected =
false

  return result;
}

/*  0: South
 *  1: East
 *  2: North
 *  3: West
 *  4: Shake - [priority]
*/
int8_t DataReader::computeAndSendAction() {
  readAverageSample();     // Read and parse average sample
  readAccelerationAxis();  // Parse the average value to rotation axis

  int8_t result = -1;

  if (detectShake())
    result = 4;  // SHAKE
  else if (accelerationAxis.X == 1)
```

```
    result = 3;  // WEST
  else if (accelerationAxis.X == -1)
    result = 1;  // EAST
  else if (accelerationAxis.Y == 1)
    result = 2;  // NORTH
  else if (accelerationAxis.Y == -1)
    result = 0;  // SOUTH
  return result;
}

// Check if a value is inside an interval, otherwise return the passed edge of
the interval
int DataReader::ClampValue(int valueToClamp, int minValue, int maxValue) const {
  if (valueToClamp >= maxValue) return maxValue;
  if (valueToClamp <= minValue) return minValue;
  return valueToClamp;
}

/*=======DEBUG=======*/
void DataReader::readAndPlotAverageSample() {
  readAverageSample();

  Serial.print("X:");
  Serial.print(avgAccDataSample.X);
  Serial.print(",Y:");
  Serial.println(avgAccDataSample.Y);
}

void DataReader::printAverageSampleDirection() {
  readAverageSample();  // Read and parse average sample
  readAccelerationAxis();   // Parse the average value to rotation axis

  // HORIZONTAL
  if (accelerationAxis.X == 1)
    Serial.print("LEFT");
  else if (accelerationAxis.X == -1)
    Serial.print("RIGHT");
  else Serial.print("No rotation");

  // VERTICAL
  Serial.print(" | ");
  if (accelerationAxis.Y == 1)
    Serial.println("UP");
  else if (accelerationAxis.Y == -1)
    Serial.println("DOWN");
```

```
  else Serial.println("No rotation");
}
```

DataReader.h

```
// Include guards, if the header is not defined already, define it.
// I.E. It might be called from multiple locations, thus including it multiple
times.
#ifndef DATAREADER_H
#define DATAREADER_H

#include "Wire.h"
#include "I2Cdev.h"
#include "MPU6050.h"

struct Vector2 {
  // 8 bits of memory for each variable for a range of [-128:127]
  int8_t X = 0;
  int8_t Y = 0;
};

// unsigned version of Vector2 with a range of [0:255]
struct uVector2 {
  byte X = 0;
  byte Y = 0;
};

class DataReader {
private:
  const int16_t samplingDelay = 50;                       // Deplay between
samples in [ms]
  const int8_t samplingCount = 3;                         // Each X samples are
averaged into a single one for better outputs
  const int8_t directionErrorThreshold = 25;              // Allowed threshold
around each step to convert sensor data to direction
  const int16_t directionUpperStep = 185;                 // Upper conversion
step from sensor data to direction
  const int16_t directionMiddleStep = 126;                // Middle conversion
step from sensor data to direction
  const int8_t directionLowerStep = 60;                   // Lower conversion
step from sensor data to direction
  int16_t ax, ay;                                         // variables used to
read the sensor rotation
  int16_t rz;                                             // variable used to
read the sensor acceleration
```

```
  uVector2 avgAccDataSample;                              // Average sample
values for every *samplingCount* samples
  Vector2 accelerationAxis;                               // Each axis can have
as value [-1/0/1];
  int16_t avgRotDataSample;                               // Average sample value
for every *samplingCount* samples
  int16_t previousAvgAccDataSample;                       // Previous stored
avgAccDataSample
  const int16_t accDifferenceThreshold = 40;              // Threshold for the
shake acceleration spike height | This can be a bit over half the signal, that
means it goes below and under, not from middle to 1 limit
  const int8_t accSpikeFlagResetThreshold = 10;           // Threshold for
reseting the acceleration spike already registered flag.
  bool accSpikeAlreadyDetected = false;                   // Flag to allow
registering a spike of acceleration. Resets near 0 crossing
  int8_t accDifferenceFlags[2] = { 0, 0 };                // How many times the
shake threshold has been hit left and right
  const int8_t accDifferenceFlagThreshold[2] = { 2, 2 };  // How many times the
shake threshold should be hit to flag a shake event

  /*=======DEBUG=======*/
  void readAndParseSampleData();
  int8_t parseAccSampleToDirection(int16_t sampleValue);

public:
  MPU6050 mpu;  // Define instance of the class to read the sensor
  DataReader();

  void initialize();               // Initialize components and modules
  void readAverageSample();        // Read [samplingCount] samples and average
them
  Vector2 readAccelerationAxis();  // Parse the acceleration axis to a direction
  bool detectShake();              // Check if there was a shake action in the
samples
  int8_t computeAndSendAction();   // Compute rotation and send action through
bluetooth

  int ClampValue(int valueToClamp, int minValue, int maxValue) const;

  /*=======DEBUG=======*/
  void readAndPlotAverageSample();     // Display the average value as to plot it
in the serial plotter
  void printAverageSampleDirection();  // Display the average sample direction
mapping
};
```

```
#endif  // DATAREADER_H
// End the "if not defined" from the top row
```

Game screen

screen.ino

```
// com5 = uno - master - 0025:00:005ED5
#include "gamemanager.h"

GameManager* gm;  // Global GameManager object

void setup() {
  Serial.begin(9600);      // Start serial output for debugging
  gm = new GameManager();  // Define GameManager object
  gm->Init();              // Initialize the GameManager object
}

void loop() {
  gm->Main(); // Main loop of the GameManager
}
```

gamemanager.cpp

```
#include "gamemanager.h"

// North - East - South - West
const byte head[4][8] = {
  { 0b00000,
    0b00000,
    0b01110,
    0b10101,
    0b10101,
    0b11111,
    0b11111,
    0b01110 },
  { 0b00000,
    0b01110,
    0b10011,
    0b11111,
    0b11111,
    0b10011,
    0b01110,
```

```
    0b00000 },
  { 0b01110,
    0b11111,
    0b11111,
    0b10101,
    0b10101,
    0b01110,
    0b00000,
    0b00000 },
  { 0b00000,
    0b01110,
    0b11001,
    0b11111,
    0b11111,
    0b11001,
    0b01110,
    0b00000 }
};
// Body
const byte body[8] = {
  0b00000,
  0b11111,
  0b11111,
  0b11111,
  0b11111,
  0b11111,
  0b11111,
  0b00000
};
// Fruits
const byte fruits[3][8] = {
  { 0b00000,
    0b00001,
    0b00011,
    0b00011,
    0b00111,
    0b11110,
    0b01100,
    0b00000 },
  { 0b00011,
    0b00100,
    0b01110,
    0b11111,
    0b11111,
    0b11111,
```

```
    0b01110,
    0b00000 },
  { 0b01110,
    0b00100,
    0b01110,
    0b11111,
    0b11111,
    0b11111,
    0b01110,
    0b00000 }
};

GameManager::GameManager()
  : lcd(7, 6, 5, 4, 3, 2),  // LCD can't be Initialized otherwise because it has
no default constructor same with bt serial
    BTSerial(10, 11),       //RX, TX
    wormManager(new Worm()) {

  pinMode(HC05STATE, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  for (int i = 0; i < 4; i++)
    for (int j = 0; j < 16; j++)
      lcdPositionMapping[i * 16 + j] = i * 16 + j;  // Instantiate each element
of the vector with its index

  // Initialize the library by associating any needed LCD interface pin with the
Arduino pin number
  // const int rs = 12, en = 11, d4 = 5, d5 = 4, d6 = 3, d7 = 2;
}

void GameManager::Init() const {
  lcd.begin(16, 4);      // Initialize lcd shield
  BTSerial.begin(9600);  // Make sure both HC-05 modules are set to same baud
rate

  lcd.createChar(0, body);  // Create the snake body special character which will
remain constant on the character value 1

  if (EEPROM.read(EEPROM_SCORE_ADDR) > 64)  // Check if the highscore set in the
memory is 64 (max score)
    EEPROM.put(EEPROM_SCORE_ADDR, 0);       // If it is, reset it to 0
}

void GameManager::Main() {
```

```
  if (tonePlaying && millis() - toneStartTime >= 200) {  // Check if the buzzer
is on, and if the time has passed
    StopBuzzer();                                        // stop buzzer
    tonePlaying = false;                                 // mark buzzer boolean
as false
  }

  if (digitalRead(HC05STATE)) {                        // If the HC-05 is
connected
    String received = BTSerial.readStringUntil('\n');  // Read incomming bits
until newline
    long command = received.toInt();                   // Parse the string bits
into int

    // Redraw screen frame
    if (isMenu)
      ShowMenu();
    else if (isPaused)
      ShowPause();
    else if (isLost)
      ShowLose();
    else if (isWon)
      ShowWin();
    else
      RedrawScreen();

    if (shouldTriggerBuzzerNextFrame) {      // Should the buzzer be triggered
this frame
      TriggerBuzzer();                       // Trigger buzzer
      shouldTriggerBuzzerNextFrame = false;  // Reset flag of nextFrameTrigger
    }

    // Check wether the string is a "4" = SHAKE or a "0|1|2|3" = "N|E|S|W"
movement type of command.
    if (received.length() - 1) {  // Check if the received string contains only
newline or something else as well
      if (command == 4) {         // SHAKE command
        if (isMenu || isLost || isWon)
          StartGame();  // Start/Restart game
        else if (isPaused)
          Resume();    // Resume game if paused
        else Pause();  // Pause game if playing

      } else if (command >= 0 && command < 4) {  // Check if the parsed value is
an actual command
```

```
        // MOVEMENT command
        if (!isPaused && !isMenu) {
          int8_t currentOrientation = wormManager->GetHeadSegment()-
>GetOrientation();  // Get orientation of the head

          // Check if this is a legal move
          if (!(abs(currentOrientation - command) == 2  // The difference is
exactly 2 for opposite directions
                && currentOrientation > -1))            // The currentOrientation
is not -1 always for head segment
          {
            bool canMoveForward = !wormManager-
>IsNextLocationOccupied(command);  // Check if the segment in front of the head
is occupied by a body part
            if (canMoveForward) {
              bool ateFruit = wormManager->CheckForFruit(fruitLocation,
command);  // Check if the next segment is a fruit
              if (ateFruit) {
                currentScore++;                       // Increase score if the
snake ate the fruit
                if (currentScore == 64)               // If the score is the
maximum (64)
                  WinGame();                          // Win the game
                else SpawnFruit();                    // Respawn fruit
                shouldTriggerBuzzerNextFrame = true;  // Trigger buzzer on eating
fruit
              }
              wormManager->MoveWormTowardsDirection(command);  // Move the snake
1 segment in front
            } else
              LoseGame();  // If the next segment was a body part, lose the game
          }
        }
      }
    }
  } else
    WaitForConnection();  // Make this take it's state based on the state pin

  delay(250);
}

void GameManager::StartGame() {
  currentScore = 0;  // Reset current round's score
  isWon = false;     // Not won yet
  isLost = false;    // Not lost yet
```

```
  isMenu = false;    // Not menu anymore
  SpawnFruit();      // Spawn the first fruit
  RedrawScreen();    // Update the frame
}

void GameManager::WinGame() {
  if (EEPROM.read(EEPROM_SCORE_ADDR) < currentScore)  // Check if the new score
is higher than the memorized score
    EEPROM.put(EEPROM_SCORE_ADDR, currentScore);      // Save the new highscore

  isWon = true;  // Set win boolean as true
}

void GameManager::LoseGame() {
  if (EEPROM.read(EEPROM_SCORE_ADDR) < currentScore)  // Check if the new score
is higher than the memorized score
    EEPROM.put(EEPROM_SCORE_ADDR, currentScore);      // Save the new highscore

  delete wormManager;        // Delete the worm object
  wormManager = new Worm();  // Reinstantiate the worm object
  isLost = true;             // Set the lose boolean as true
}

void GameManager::Resume() {
  isMenu = false;    // the game is playing, no longer menu
  isPaused = false;  // the game is playing, no longer paused
}

void GameManager::Pause() {
  isMenu = false;   // the game is paused, no longer menu
  isPaused = true;  // the game is paused
}

void GameManager::ShowPause() {
  char scoreStr[6];                       // instantiate array where the score
string will be concatenated
  itoa(currentScore, scoreStr, 10);  // transform the int to a char and save it
in scoreStr

  char result[15] = "SCOREBOARD:";
  if (currentScore < 10)
    strcat(result, "0");     // Add a 0 in front of the number if it has 1 digit
only
  strcat(result, scoreStr);  // Concatenate the score to the final result
```

```
  strcat(result, "/64");     // The snake needs to be 16x4 = 64 segments long to
win (that's the max highscore)


  lcd.clear();          // Clear LCD
  lcd.setCursor(1, 0);  // Set cursor to ((16 - length_of_string)/2 ; row)
  lcd.write("<<--Shake-->>");
  lcd.setCursor(7, 1);
  lcd.write("to");
  lcd.setCursor(3, 2);
  lcd.write("--Resume--");
  lcd.setCursor(0, 3);
  lcd.write(result);
}

void GameManager::ShowMenu() {
  // Show start and highscore

  int8_t highscoreValue = EEPROM.read(EEPROM_SCORE_ADDR);  // Read the current
highscore stored in the memory
  char highscoreStr[6];                                    // instantiate array
where the score string will be concatenated
  itoa(highscoreValue, highscoreStr, 10);                  // transform the int
to a char and save it in highscoreStr

  char result[15] = "Highscore:";
  if (highscoreValue < 10)
    strcat(result, "0");         // Add a 0 in front of the number if it has 1
digit only
  strcat(result, highscoreStr);  // Concatenate the score to the final result
  strcat(result, "/64");         // The snake needs to be 16x4 = 64 segments long
to win (that's the max highscore)

  lcd.clear();          // Clear LCD
  lcd.setCursor(1, 0);  // Set cursor to ((16 - length_of_string)/2 ; row)
  lcd.write("<<--Shake-->>");
  lcd.setCursor(7, 1);
  lcd.write("to");
  lcd.setCursor(3, 2);
  lcd.write("--START--");
  lcd.setCursor(1, 3);
  lcd.write(result);
}

void GameManager::ShowWin() const {
```

```
  lcd.clear();          // Clear LCD
  lcd.setCursor(4, 0);  // Set cursor to ((16 - length_of_string)/2 ; row)
  lcd.write("YOU WON!");
  lcd.setCursor(1, 1);
  lcd.write("<<--Shake-->>");
  lcd.setCursor(7, 2);
  lcd.write("to");
  lcd.setCursor(3, 3);
  lcd.write("--Restart--");
}

void GameManager::ShowLose() const {
  char scoreStr[6];                       // doesn't work with pointer :(
  itoa(currentScore, scoreStr, 10);  // int , char to store result, base

  char result[16] = "FINALSCORE:";
  if (currentScore < 10)
    strcat(result, "0");     // Add a 0 in front of the number if it has 1 digit
only
  strcat(result, scoreStr);  // Concatenate the score to the final result
  strcat(result, "/64");     // The snake needs to be 16x4 = 64 segments long to
win (that's the max highscore)

  lcd.clear();          // Clear LCD
  lcd.setCursor(1, 0);  // Set cursor to ((16 - length_of_string)/2 ; row)
  lcd.write("<<--Shake-->>");
  lcd.setCursor(7, 1);
  lcd.write("to");
  lcd.setCursor(3, 2);
  lcd.write("--Restart--");
  lcd.setCursor(0, 3);
  lcd.write(result);
}

void GameManager::RedrawScreen() const {
  lcd.clear();                                          // Clear LCD
  Segment* wormSegment = wormManager->GetHeadSegment();  // Get first segment -
the head

  while (wormSegment != nullptr)
{                                                      // Iterate while there is
no segment left
    uVector2 location = wormSegment-
>GetLocation();                                     // Get the location of the
current segment
```

```
    WriteCharacterFromSegmentAt(wormSegment->GetOrientation(), location.X,
location.Y);  // Display the segment on the screen
    wormSegment = wormSegment-
>GetNextSegment();                                        // Get the next
segment
  }

  lcd.setCursor(fruitLocation.X, fruitLocation.Y);  // Set location of the fruit
  lcd.write((uint8_t)2);                            // Draw the fruit stored in
the special character 2
}

void GameManager::TriggerBuzzer() {
  StartBuzzer();             // Start buzzer
  tonePlaying = true;        // Set the buzzerPlaying to true
  toneStartTime = millis();  // Store the starting time of the buzzer
}

void GameManager::StartBuzzer() {
  // Setup Timer1 in CTC mode to toggle OC1A (Digital pin 9)
  TCCR1A = 0;  // Clear any previous settings
  TCCR1B = 0;  // Clear any previous settings

  // CTC mode
  TCCR1B |= (1 << 3);  // (1 << WGM12);

  // Toggle OC1A on Compare Match
  TCCR1A |= (1 << 6);  // (1 << COM1A0);

  // Prescaler 8 → 16MHz / 8 = 2MHz
  TCCR1B |= (1 << 1);  // (1 << CS11);

  // 600 Hz -> OCR1A = (2MHz / (2 * 600)) - 1 = 1666
  OCR1A = 1666;
}

void GameManager::StopBuzzer() {
  // Stop Timer1 and clean up
  TCCR1A = 0;
  TCCR1B = 0;
  digitalWrite(BUZZER_PIN, LOW);  // Turn the buzzer off
}

void GameManager::SpawnFruit() {
```

```
  uVector2 randomEmptyLocation = GetEmptyLocation();  // Get a random empty
location from the map
  fruitLocation.X = randomEmptyLocation.X;            // Store the empty location
as the next fruit location
  fruitLocation.Y = randomEmptyLocation.Y;

  lcd.createChar(2, fruits[random(3)]);  // Create a new random fruit as the next
fruit character stored in the special character 2
}

uVector2 GameManager::GetEmptyLocation() const {
  Vector* lcdMapping = new Vector(16 * 4, true);  // Instantiate a new vector
that maps the entire LCD

  Segment* wormSegment = wormManager->GetHeadSegment();  // Get first segment -
the head

  while (wormSegment != nullptr) {                       // Iterate all segments
until no follower remains
    uVector2 location = wormSegment->GetLocation();      // Get the currrent
segment's location
    lcdMapping->removeAt(location.Y * 16 + location.X);  // Clear the occupied
cells from the mapping
    wormSegment = wormSegment->GetNextSegment();         // Get next segment
  }

  int randomIndex = random(lcdMapping->getLength());  // Pick a random index left
from the vector
  int randomCell = lcdMapping->valueAt(randomIndex);  // Pick the value of that
index

  delete lcdMapping;                                  // Free the space allocated
by the vector
  return uVector2(randomCell % 16, randomCell / 16);  // Division to 16 gives the
result = Y and reminder = X
}

void GameManager::WriteCharacterFromSegmentAt(int8_t orientation, int8_t x,
int8_t y) {
  uint8_t characterToWrite = 1;  // Default head

  // 0=N ; 1=E ; 2=S ; 3=W
  switch (orientation) {
    case -1:  // Body segment
      characterToWrite = 0;
```

```
      // Body character is constant and already created. Just display it.
      break;
    case 0:  // Head facing NORTH
      lcd.createChar(1, head[0]);
      break;
    case 1:  // Head facing EAST
      lcd.createChar(1, head[3]);
      break;
    case 2:  // Head facing SOUTH
      lcd.createChar(1, head[2]);
      break;
    case 3:  // Head facing WEST
      lcd.createChar(1, head[1]);
      break;
  }
  lcd.setCursor(x, y);          // The location is preset from the arguments
  lcd.write(characterToWrite);  // The character to display is chosen above
}

void GameManager::WaitForConnection() {
  lcd.clear();          // Clear LCD
  lcd.setCursor(4, 0);  // Set cursor to ((16 - length_of_string)/2 ; row)
  lcd.write("waiting");
  lcd.setCursor(6, 1);
  lcd.write("for");
  lcd.setCursor(3, 2);
  lcd.write("connection");
  lcd.setCursor(6, 3);
  char* loadingMsg = "";            // Instantiate the connection string loading
animation char array
  switch (connectionStringState) {  // Based on the current state pick a
    case 0:
      loadingMsg = "";
      break;
    case 1:
      loadingMsg = ".";
      break;
    case 2:
      loadingMsg = "..";
      break;
    case 3:
      loadingMsg = "...";
      break;
    default:
      connectionStringState = 0;
```

```
      loadingMsg = "";
      break;
  }
  lcd.write(loadingMsg);    // Display the message on the LCD
  connectionStringState++;  // Increase the state
  delay(200);               // Delay between loading sequences
}
```

gamemanager.h

```
#ifndef GAMEMANAGER_H
#define GAMEMANAGER_H
#endif

#include "worm.h"
#include "vector.h"

#include <EEPROM.h>
#include <LiquidCrystal.h>
#include <SoftwareSerial.h>

#include <stdint.h>  // for 'int8_t'

#define HC05STATE 8
#define BUZZER_PIN 9
#define EEPROM_SCORE_ADDR 0

class GameManager {
private:
  LiquidCrystal lcd;
  SoftwareSerial BTSerial;
  Worm* wormManager;

  uVector2 fruitLocation;
  uint8_t connectionStringState = 0;
  int currentScore;
  bool isPaused = false,
       isMenu = true,
       isLost = false,
       isWon = false;
  int lcdPositionMapping[16 * 4];
  bool tonePlaying = false;
  bool shouldTriggerBuzzerNextFrame = false;
  unsigned long toneStartTime = 0;
```

```
public:
  GameManager();
  void Init() const;
  void Main();
  void StartGame();
  void Resume();
  void Pause();
  void WinGame();
  void LoseGame();
  void ShowPause();
  void ShowMenu();
  void ShowWin() const;
  void ShowLose() const;
  void RedrawScreen() const;

  void TriggerBuzzer();
  void StartBuzzer();
  void StopBuzzer();
  void SpawnFruit();
  uVector2 GetEmptyLocation() const;
  void WriteCharacterFromSegmentAt(int8_t orientation, int8_t x, int8_t y);
  void WaitForConnection();
};
```

worm.cpp

```
#include "worm.h"

Worm::Worm() {
  wormHead = new Segment(true);
  uVector2 headLocation = wormHead->GetLocation();
  wormHead->SetNextSegment(new Segment(headLocation.X - 1, headLocation.Y));
}

void Worm::IncreaseWormSize() {
  Segment* currentSegment = wormHead;                  // Start iteration from
head
  while (currentSegment->GetNextSegment() != nullptr)  // Check if the current
segment has a follow up segment
    currentSegment = currentSegment->GetNextSegment();

  currentSegment->SetNextSegment(new Segment(false));  // Create new segment at
the end
```

```
}

void Worm::MoveWormTowardsDirection(uint8_t direction) {
  uVector2 previousLocation, auxLocation, newLocation;  // All 3 Initialized by
the default struct constructor with -1

  Segment* currentSegment = wormHead;             // Start iteration from head
  while (currentSegment != nullptr) {             // Check if the current segment
has a follow up segment
    auxLocation = currentSegment->GetLocation();  // Save current segment
location
    // TODO: Check if you can move there
    if (previousLocation.X == -1 || previousLocation.Y == -1) {
      // First move - HEAD

      // Check if the head doesn't try to turn 180 deg
      int8_t currentOrientation = currentSegment->GetOrientation();
      if (abs(currentOrientation - direction) == 2  // The difference is exactly
2 for opposite directions
          && currentOrientation > -1)               // The currentOrientation is
not -1 always for head segment
        return;                                     // Opposites detected - Can't
fullfill request

      currentSegment->SetOrientation(direction);
      newLocation = GetNextLocation(auxLocation, direction);

      previousLocation = currentSegment->GetLocation();

      currentSegment->SetLocation(newLocation);
    } else {
      // Rest of body segments
      currentSegment->SetLocation(previousLocation);

      previousLocation = auxLocation;
    }

    currentSegment = currentSegment->GetNextSegment();  // Advance to the next
segment
  }
}

uVector2 Worm::GetNextLocation(uVector2 currentLocation, uint8_t direction) {
  uVector2 newLocation = currentLocation;
  switch (direction) {
```

```
    case 0:  //NORTH
      newLocation.Y -= 1;
      break;
    case 1:  //EAST
      newLocation.X += 1;
      break;
    case 2:  //SOUTH
      newLocation.Y += 1;
      break;
    case 3:  //WEST
      newLocation.X -= 1;
      break;
  }

  // Change these to a clamp function on screen, where screen size is Initialized
at start
  newLocation.X = newLocation.X < 0 ? 15 : newLocation.X;
  newLocation.X = newLocation.X > 15 ? 0 : newLocation.X;

  newLocation.Y = newLocation.Y < 0 ? 3 : newLocation.Y;
  newLocation.Y = newLocation.Y > 3 ? 0 : newLocation.Y;
  return newLocation;
}

bool Worm::IsNextLocationOccupied(int8_t orientation) {
  uVector2 nextLocation = GetNextLocation(wormHead->GetLocation(), orientation);
  uVector2 currentLocation;

  Segment* currentSegment = wormHead;                 // Start iteration from
head
  while (currentSegment != nullptr) {                 // Check if the current
segment has a follow up segment
    currentLocation = currentSegment->GetLocation();  // Save current segment
location
    if (currentLocation.X == nextLocation.X && currentLocation.Y ==
nextLocation.Y)
      return true;

    currentSegment = currentSegment->GetNextSegment();
  }

  return false;
}
```

```
bool Worm::CheckForFruit(uVector2 fruitLocation, int8_t orientation) {
  uVector2 nextLocation = GetNextLocation(wormHead->GetLocation(), orientation);
  if (fruitLocation.X == nextLocation.X && fruitLocation.Y == nextLocation.Y) {
    IncreaseWormSize();
    return true;
  }
  return false;
}


Segment* Worm::GetHeadSegment() const {
  return wormHead;
}
```

worm.h

```
// Include guards, if the header is not defined already, define it.
// I.E. It might be called from multiple locations, thus including it multiple
times.
#ifndef WORM_H
#define WORM_H
#endif

#include "segment.h"
#include "Arduino.h"

#include <stdint.h>  // for 'int8_t'

class Worm {
private:
  Segment* wormHead;
public:
  Worm();
  void IncreaseWormSize();
  // 0-N | 1-E | 2-S | 3-W
  void MoveWormTowardsDirection(uint8_t direction);
  uVector2 GetNextLocation(uVector2 currentLocation, uint8_t direction);
  bool IsNextLocationOccupied(int8_t orientation);
  bool CheckForFruit(uVector2 fruitLocation, int8_t orientation);
  Segment* GetHeadSegment() const;
  // uint8_t GetRandomFreeCell() const; // move into game manager
};
```

segment.cpp

```cpp
#include "segment.h"

Segment::Segment(bool isHead) {
  location.X = 8;
  location.Y = 2;
  orientation = isHead ? 1 : -1;  // instead of 1 stay still until you get input
  nextSegment = nullptr;
}

Segment::Segment(int x, int y) {
  location.X = x;
  location.Y = y;
  orientation = -1; // Never used for head
  nextSegment = nullptr;
}

int8_t Segment::GetOrientation() const {
  return orientation;
}

void Segment::SetOrientation(int8_t newOrientation) {
  if (orientation != -1)
    orientation = newOrientation;
}

uVector2 Segment::GetLocation() const {
  return location;
}

void Segment::SetLocation(uVector2 newLocation) {
  location.X = newLocation.X;
  location.Y = newLocation.Y;
}


Segment* Segment::GetNextSegment() const {
  return nextSegment;
}

void Segment::SetNextSegment(Segment* newSegment) {
  nextSegment = newSegment;
}
```

segment.h

```
// Include guards, if the header is not defined already, define it.
// I.E. It might be called from multiple locations, thus including it multiple
times.
#ifndef SEGMENT_H
#define SEGMENT_H
#endif

#include <stdint.h>   // for 'int8_t'

// This should be imported from a different utils header and also removed from
DataReader.h
struct uVector2 {
  int8_t X = 0;
  int8_t Y = 0;

  uVector2() : X(-1), Y(-1) {}
  uVector2(int8_t x, int8_t y) : X(x), Y(y) {}
};

class Segment {
private:
  uVector2 location;
  int8_t orientation;    // Orientation of the body: -1 | Orientation of the
head: 0=N ; 1=E ; 2=S ; 3=W
  Segment* nextSegment;  // Pointer towards the next body segment. nullptr if
this is the tail
public:
  Segment(bool isHead);
  Segment(int x, int y);
  int8_t GetOrientation() const;
  void SetOrientation(int8_t newOrientation);
  uVector2 GetLocation() const;
  void SetLocation(uVector2 newLocation);
  Segment* GetNextSegment() const;
  void SetNextSegment(Segment* newSegment);
};
```

Vector.cpp

```
#include "vector.h"

Vector::Vector(int len) {
  length = len;
  values = new int[len];
```

```
}

Vector::Vector(int len, int InitialValue) {
  length = len;
  values = new int[len];
  for (int i = 0; i < len; ++i)
    values[i] = InitialValue;
}

Vector::Vector(int len, bool useIndex) {
  length = len;
  values = new int[len];
  if (useIndex)
    for (int i = 0; i < len; ++i)
      values[i] = i;
}

int Vector::getLength() const {
  return length;
}

void Vector::push(int value) {
  int* aux = new int[length];
  for (int i = 0; i < length; i++)
    aux[i] = values[i];

  delete[] values;

  values = new int[++length];
  for (int i = 0; i < length - 1; i++)
    values[i] = aux[i];
  values[length - 1] = value;

  delete[] aux;
}

int Vector::pop() {
  return removeAt(length - 1);
}

int Vector::removeAt(int index) {
  if (index >= length) return;

  int removedValue;
  int* aux = new int[length];
```

```
  for (int i = 0; i < length; i++) {
    aux[i] = values[i];
    if (i == index)
      removedValue = values[i];
  }

  delete[] values;

  values = new int[--length];
  for (int i = 0; i < length; i++)
    values[i] = aux[i < index ? i : i + 1];  // Step over the removed value

  delete[] aux;
  return removedValue;
}

int Vector::valueAt(int index) {
  return values[index];
}
```

Vector.h

```
#ifndef VECTOR_H
#define VECTOR_H
#endif

class Vector {
private:
  int length;
  int *values;
public:
  Vector(int len);
  Vector(int len, int InitialValue);
  Vector(int len, bool useIndex);
  int getLength() const;
  void push(int value);
  int pop();
  int removeAt(int index);
  int valueAt(int index);
};
```